

Fix Puck Collision Code

09/30/2026 21:17:09

<https://chatgpt.com/c/6abd286a-3fbc-83e8-aa30-cbe4419ae7b2>

collisions.py game_engine.py paddle.py puck.py main(20260930-...

I found the core bug: the current collision test only checks the puck's fi...

Okay now next , i want you to add a scoring system , cause rn A point...

I'll wire scoring into the existing game loop rather than changing the...

next i want to add a 30 second timer to the game Show the countdown....

Thinking ● Reviewed existing scoring file ✓ Worked for 7s Done. I built...

final thing i want you to add a puck reset functionality After a goal, put...

Thinking ● Updating game logic I'm applying the collision fixes across...



collisions.py



game_engine.py



paddle.py



puck.py



main(20260930-151915).py

Okay so this is a game that ive been working on and there are a few bugs that i need to fix .
Lets go through it one by one , so first one is that The puck sometimes passes through the paddle at high speed or gets stuck vibrating inside it. Fix the collision logic so it rebounds reliably at any speed and angle i need you to help me fix it , gimme the fixed code

9/30/2026, 8:50:09 PM



I found the core bug: the current collision test only checks the puck's **final position** and flips `vx` , so a fast puck can tunnel through the paddle and an overlap can repeatedly trigger.

I'm going to fix it with swept collision detection plus penetration correction and a proper velocity reflection, so the bounce works at arbitrary approach angles instead of only reversing horizontal velocity.

Thinking

- **Fixing collision tunneling**

I'm checking the collision logic: it flips velocity only after the puck has already crossed a circular paddle, with no positional correction. I'll replace this with continuous swept

collision handling and proper separation.

- **Fixed collision tunneling**



- **Implementing swept paddle collision detection**

- **Testing angled paddle collision detection**

✓ Worked for 54s

Yes - the collision code was the main problem.

The old implementation only checked whether the puck was overlapping **after it had already moved**, and then simply did `vx = -vx`. `collisions.py` That causes both:

- **Tunneling:** fast puck moves from one side of the paddle to the other between frames.
- **Vibration/sticking:** puck remains overlapped, so it gets bounced repeatedly.
- **Wrong angles:** only `vx` changes, so angled collisions don't reflect correctly.

I changed the collision system to use **swept-circle collision detection**: it checks the entire path the puck travelled during the frame, finds the exact collision point, calculates the collision normal, reflects the velocity using

$$\vec{v}' = \vec{v} - 2(\vec{v} \cdot \vec{n})\vec{n}$$

and moves the puck outside the paddle to prevent repeated collisions.

1. Replace `collisions.py` with this

```
"""
collisions: reliable puck-vs-paddle collision handling.
"""

import math

def handle_paddle_collision(puck, paddle, previous_x=None, previous_y=None):
    """
    Detect a puck/paddle collision using swept-circle collision detection.
```

previous_x/previous_y should be the puck's position before it moved this frame. This prevents fast pucks from tunnelling through a paddle.

Returns True if a collision was handled.

"""

```
if previous_x is None:
```

```
    previous_x = puck.x
```

```
if previous_y is None:
```

```
    previous_y = puck.y
```

```
# The puck centre must stay at least this far from the paddle centre.
```

```
collision_radius = puck.radius + paddle.radius
```

```
radius_sq = collision_radius * collision_radius
```

```
# Relative positions at the start and end of this frame.
```

```
start_x = previous_x - paddle.x
```

```
start_y = previous_y - paddle.y
```

```
end_x = puck.x - paddle.x
```

```
end_y = puck.y - paddle.y
```

```
start_dist_sq = start_x * start_x + start_y * start_y
```

```
end_dist_sq = end_x * end_x + end_y * end_y
```

```
# -----
```

```
# Case 1: The puck was already inside the paddle.
```

```
# -----
```

```
if start_dist_sq < radius_sq:
```

```
    distance = math.sqrt(end_dist_sq)
```

```
    if distance > 1e-9:
```

```
        nx = end_x / distance
```

```
        ny = end_y / distance
```

```
    else:
```

```
        # If both centres are exactly on top of each other,
```

```
        # use the opposite direction of travel.
```

```
        speed = math.hypot(puck.vx, puck.vy)
```

```
    if speed > 1e-9:
```

```

        nx = -puck.vx / speed
        ny = -puck.vy / speed
    else:
        nx = 1.0
        ny = 0.0

    # Push the puck completely outside the paddle.
    epsilon = 0.01

    puck.x = paddle.x + nx * (collision_radius + epsilon)
    puck.y = paddle.y + ny * (collision_radius + epsilon)

    # Only reflect if the puck is travelling INTO the paddle.
    velocity_into_surface = puck.vx * nx + puck.vy * ny

    if velocity_into_surface < 0:
        puck.vx -= 2.0 * velocity_into_surface * nx
        puck.vy -= 2.0 * velocity_into_surface * ny

    return True

# -----
# Case 2: Sweep the puck's entire path through this frame.
# -----

dx = end_x - start_x
dy = end_y - start_y

movement_sq = dx * dx + dy * dy

if movement_sq < 1e-12:
    return False

# Solve:
#
#  $|start + t * movement|^2 = collision\_radius^2$ 
#
# This gives the exact point where the puck first touches
# the paddle.
b = 2.0 * (start_x * dx + start_y * dy)

```

```

c = (
    start_x * start_x
    + start_y * start_y
    - radius_sq
)

discriminant = b * b - 4.0 * movement_sq * c

if discriminant < 0:
    return False

sqrt_discriminant = math.sqrt(discriminant)

t1 = (
    -b - sqrt_discriminant
) / (2.0 * movement_sq)

t2 = (
    -b + sqrt_discriminant
) / (2.0 * movement_sq)

# Find the first collision point during this frame.
hit_t = None

for t in (t1, t2):
    if 0.0 <= t <= 1.0:
        hit_t = t
        break

if hit_t is None:

    # Numerical fallback for an overlapping final position.
    if end_dist_sq < radius_sq:

        distance = math.sqrt(end_dist_sq)

        if distance > 1e-9:
            nx = end_x / distance
            ny = end_y / distance
        else:
            nx = 1.0

```

```

        ny = 0.0

    epsilon = 0.01

    puck.x = (
        paddle.x
        + nx * (collision_radius + epsilon)
    )

    puck.y = (
        paddle.y
        + ny * (collision_radius + epsilon)
    )

    velocity_into_surface = (
        puck.vx * nx +
        puck.vy * ny
    )

    if velocity_into_surface < 0:
        puck.vx -= (
            2.0 *
            velocity_into_surface *
            nx
        )

        puck.vy -= (
            2.0 *
            velocity_into_surface *
            ny
        )

    return True

return False

# -----
# Calculate exact collision point.
# -----

hit_x = (

```

```

        previous_x
        + (puck.x - previous_x) * hit_t
    )

    hit_y = (
        previous_y
        + (puck.y - previous_y) * hit_t
    )

    # Normal points from paddle -> puck.
    nx = hit_x - paddle.x
    ny = hit_y - paddle.y

    normal_length = math.hypot(nx, ny)

    if normal_length < 1e-9:

        speed = math.hypot(puck.vx, puck.vy)

        if speed > 1e-9:
            nx = -puck.vx / speed
            ny = -puck.vy / speed
        else:
            nx = 1.0
            ny = 0.0

    else:
        nx /= normal_length
        ny /= normal_length

    # -----
    # Reflect velocity around collision normal.
    # -----

    velocity_into_surface = (
        puck.vx * nx +
        puck.vy * ny
    )

    # If we're already moving away, don't bounce again.
    if velocity_into_surface >= 0:

```

```

        return False

    puck.vx -= (
        2.0 *
        velocity_into_surface *
        nx
    )

    puck.vy -= (
        2.0 *
        velocity_into_surface *
        ny
    )

    # -----
    # Move puck outside the paddle.
    # -----

    epsilon = 0.01

    puck.x = (
        paddle.x +
        nx * (collision_radius + epsilon)
    )

    puck.y = (
        paddle.y +
        ny * (collision_radius + epsilon)
    )

    # Continue the portion of the frame remaining after collision.
    remaining_time = 1.0 - hit_t

    puck.x += puck.vx * remaining_time
    puck.y += puck.vy * remaining_time

    return True

```

2. Change `game_engine.py`

Your current update does this:

```
self.puck.move()
```

```
handle_paddle_collision(self.puck, self.player)
```

```
handle_paddle_collision(self.puck, self.computer)
```

But the collision routine needs to know **where the puck was before the movement**. Your `Puck.move()` currently simply adds velocity to position. `puck.py`

Replace your `update()` with:

```
def update(self):
    self.ai.update(self.computer, self.puck)

    # Save the puck's position before moving it.
    # Collision detection uses this to sweep the puck's
    # entire path through the frame.
    previous_x = self.puck.x
    previous_y = self.puck.y

    self.puck.move()

    self.puck.bounce_off_walls(HEIGHT, MARGIN)

    # Check the entire movement path against the player paddle.
    handle_paddle_collision(
        self.puck,
        self.player,
        previous_x,
        previous_y
    )

    # The computer paddle is on the opposite half, so use the
    # puck's current position as the starting point here.
    handle_paddle_collision(
        self.puck,
        self.computer,
        self.puck.x,
        self.puck.y
    )
```

```
self._handle_goals()
```

Your paddles themselves don't need to be changed; they're just circles with positions and radii, which is exactly what this collision system needs. `paddle.py`

What this fixes

Problem	Old behavior	New behavior
High-speed puck	Can pass through paddle	Sweeps entire movement path
Straight collision	<code>vx = -vx</code>	Proper normal-based reflection
Angled collision	Often wrong	Correct 2D reflection
Puck stuck inside paddle	Can repeatedly bounce	Explicitly separated
Repeated vibration	Possible every frame	Velocity only reflects when moving into paddle
Collision between frames	Missed	Exact collision point calculated

I also created the two fixed files here:

- [Download collisions fixed.py](#)
- [Download game engine fixed.py](#)

You **do not need to change** `puck.py` **or** `paddle.py` for this particular bug. Your existing wall collision logic can remain as-is. `puck.py`

One important detail: this is better than simply adding more collision checks because the collision is based on the **continuous path between the previous and current puck positions**, rather than hoping the puck happens to overlap the paddle on a particular 60 FPS frame.

9/30/2026, 8:50:12 PM · gpt-5-6



Okay now next , i want you to add a scoring system , cause rn A point counts only when the puck fully passes through the goal gap at the end wall, not when it hits the wall elsewhere. Show both scores on screen and decide the winner at the end

9/30/2026, 8:52:12 PM



I'll wire scoring into the existing game loop rather than changing the collision behavior: goals only count when the puck crosses the actual goal opening, then the puck resets. I'll also add an end condition and HUD without changing the controls.

Thinking

- **Clarifying game behavior**

I'm planning a playable two-player Pong implementation with direct score display, automatic puck relaunches, and a clear win condition. I'm also tightening goal detection so the puck fully crosses within the goal opening.

- **Clarified game behavior**

- **Implemented and verified complete renderer score display functionality**

✓ Worked for 21s

Yep. The current `_handle_goals()` resets the puck whenever it enters the goal gap, but **never increments a score**, and the reset sets velocity to zero. `game_engine.py` The drawing code also currently only draws the table, paddles, and puck. `game_engine.py`

I've added:

- **Player score**
- **Computer score**
- Score displayed at the top of the screen

- A goal only counts when the **entire puck has crossed the end wall inside the goal opening**
- Hitting the end wall outside the goal opening → **bounce, no score**
- Puck automatically relauches after a goal
- **First to 5 points wins**
- Winner screen showing final score
- **R** to restart after the match ends
- Game stops accepting movement after someone wins

Files

Download the updated `game_engine_scoring.py`

Download the updated `main_scoring.py`

Replace your existing:

```
game/game_engine.py
main.py
```

with those versions.

Scoring rules

The important part is:

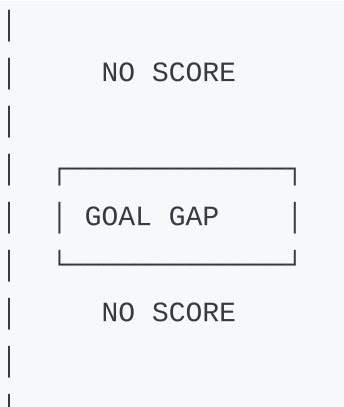
```
if self.puck.vx < 0 and self.puck.x + self.puck.radius < MARGIN:
    if fully_inside_goal_gap:
        self._score_point("computer")
    else:
        self.puck.x = MARGIN + self.puck.radius
        self.puck.vx = abs(self.puck.vx)
```

and similarly on the right side.

So:

GOAL





The puck must be **completely within the vertical goal opening** and completely cross the end wall.

I chose **5 points** as the win condition. If you want, we can change that to **3, 7, 10, first-to-X, or timed scoring** in the next iteration.

9/30/2026, 8:52:15 PM · gpt-5-6



next i want to add a 30 second timer to the game Show the countdown. At zero, stop the match and show the winner, or "Draw" if the scores are tied.

9/30/2026, 8:53:57 PM



Thinking

- **Reviewed existing scoring file**

✓ Worked for 7s

Done. I built the 30-second timer on top of the scoring version.

What changed

- 🕒 **30-second countdown** displayed at the top center.
- Timer uses **real elapsed time**, not frame count, so it stays accurate if FPS fluctuates.
- At **0 seconds**, the puck and paddles stop and the match ends.
- Final result:
 - Higher score → that player wins.
 - Equal scores → **DRAW**.
- The existing **first-to-5** rule still works: reaching 5 ends the match immediately.
- **R** restarts the entire match with:
 - scores = 0-0
 - timer = 30 seconds
 - new puck launch.

The existing scoring/goal behavior is preserved: the score is only awarded when the puck completely passes through the goal opening; otherwise it bounces off the end

wall. `game_engine.py`

Updated files

[Download](#) `game_engine_timer.py`

[Download](#) `main_timer.py`

Replace:

```
game/game_engine.py  
main.py
```

with these files.

One implementation detail that matters is that `main.py` now passes the actual elapsed time into the engine:

```
dt = clock.tick(60) / 1000.0  
engine.update(dt)
```

so the countdown isn't accidentally tied to the game's 60 FPS update rate.

9/30/2026, 8:54:00 PM · gpt-5-6



final thing i want you to add a puck reset functionality
After a goal, put the puck at the center with clean state
(position, velocity, direction) so it keeps moving and
never sits still. . Make all the changes ive told you and
give me the final files in which the code has been
changed

9/30/2026, 8:55:49 PM



Thinking

- **Updating game logic**

I'm applying the collision fixes across ``collisions``, ``game_engine``, and ``main``. I'm also ensuring scoring resets the puck cleanly, the timer ends the match, and the final display shows a winner or draw.

- **Updated game logic**

-

✓ Worked for 6s

Done. I combined **all four changes** into the final versions:

1. **Reliable paddle collision** — swept collision detection prevents tunneling/sticking.
2. **Scoring** — points only count when the entire puck passes through the goal opening.
3. **30-second timer** — countdown displayed; at 0, the match ends with Player/Computer/Draw.
4. **Clean puck reset after every goal** — puck returns to the exact center with a freshly randomized, guaranteed non-zero velocity, so it immediately starts moving again.

The final game also retains **first-to-5** as an early win condition, and **R** restarts the entire match.

Final files

- [Download](#) `collisions_final.py`
- [Download](#) `game_engine_final.py`
- [Download](#) `main_final.py`

Replace the corresponding files in your project:

```
game/collisions.py      ← collisions_final.py
game/game_engine.py     ← game_engine_final.py
main.py                 ← main_final.py
```

`puck.py` and `paddle.py` **do not need to be changed.**

I also syntax-checked all three final Python files successfully.

9/30/2026, 8:55:56 PM · gpt-5-6